

Assignment: Building "Java City"

Goal: Solve 20 practical problems by building small, connected systems.

Phase 1: The Personal ID System (Classes & Encapsulation)

Scenario: Every citizen in Java City needs a digital profile.

1. **The Profile:** Create a Citizen class. Give it a name, age, and voterID.
2. **Privacy First:** Make all fields private. Create a method setAge(int newAge). If the age is less than 0, print an error instead of saving it.
3. **The Secret Password:** Add a private String password. Create a method changePassword(String oldPass, String newPass). Only update the password if the oldPass matches the current one.
4. **Automatic ID:** Use a static int citizenCount to automatically assign a unique ID number (1, 2, 3...) to every new Citizen object created.

Phase 2: The City Transport (Inheritance)

Scenario: Java City has different types of vehicles that all share basic traits.

5. **The Blueprint:** Create a Vehicle class with speed and fuel. Add a method move() that prints "Vehicle is moving."
6. **The Upgrade:** Create a Bicycle class and a Car class that inherit from Vehicle.
7. **Specific Behavior:** Override the move() method in Bicycle to say "Pedaling at [speed] km/h" and in Car to say "Driving at [speed] km/h."
8. **The Horn:** Add a soundHorn() method to Vehicle. Make the Car override it to say "Beep Beep," but leave Bicycle alone so it uses the default "Ring Ring" (demonstrating inherited behavior).

Phase 3: The City Power Grid (Abstraction & Interfaces)

Scenario: Different buildings consume power differently.

9. **The Power Plug:** Create an interface PowerConsumer with a method consumeEnergy(int watts).
10. **The Mall vs. The House:** Create a ShoppingMall class and a TinyHouse class. Both must implement PowerConsumer.
11. **Usage Logic:** In ShoppingMall, if watts > 1000, print "High usage warning!" In TinyHouse, just print "Powering the lights."
12. **The Abstract Foundation:** Create an abstract class Building with a field address. Make ShoppingMall extend Building **and** implement PowerConsumer.

Phase 4: The Pet Shelter (Polymorphism)

Scenario: A shelter manages different animals. This teaches how to handle multiple types as one.

13. **The Animal Group:** Create a base class `Animal` with a method `makeSound()`. Create subclasses `Dog` and `Cat`.
14. **The Array Challenge:** In your `Main` class, create an `Animal[]` array. Put two `Dog` objects and two `Cat` objects inside it.
15. **The Chorus:** Loop through that array and call `.makeSound()`. Notice how the computer knows which sound to make even though the array type is just `Animal`.
16. **The Vet Check:** Create a method `checkHealth(Animal a)`. Pass a `Dog` to it, then a `Cat`. Show that one method can handle any animal.

Phase 5: The Smart Home (Composition)

Scenario: A "Smart Home" isn't a type of `Light`; it *contains* `Lights`.

17. **The Components:** Create a `LightBulb` class with `isOn` (boolean) and `toggle()` methods.
18. **The Assembly:** Create a `SmartRoom` class. Inside it, declare private `LightBulb roomLight`.
19. **The Connection:** In the `SmartRoom` constructor, initialize the `roomLight`. Create a `pressWallSwitch()` method in `SmartRoom` that calls `roomLight.toggle()`.
20. **The Expansion:** Add a `Speaker` object to the `SmartRoom`. Create a `partyMode()` method that turns on the light and plays music at the same time.

Tip:

- **Use the main method:** Create a `CityController` class with a public static void `main` to test every feature you build.